先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 16 – Deep Learning II: Biological Vision and Convolutional Networks

Chizhi Chris ZHANG

zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Three periods: pixels, feature maps, and vision tasks

Time	Main question	Calculation or application
45 min	What does a shared local filter compute?	image axes, a full 3×3 example, output size
45 min	How do channels and spatial resolution change?	parameters, pooling, receptive fields, a small CNN
45 min	How do we build and use deeper visual models?	bottlenecks, residuals, efficient convolution, task heads

Continuous reasoning

For each operation, track the input shape, output shape, learned parameters, and spatial information.

By the end

Design a small image classifier and explain why a segmentation model needs a different output.

Flattening a large image makes a dense layer expensive

An RGB image of 224×224 contains $224 \cdot 224 \cdot 3 = 150{,}528$ numbers.

$$\text{Dense layer to 1,000 units:} \quad 150{,}528(1{,}000) + 1{,}000 = 150{,}529{,}000 \text{ parameters.}$$

- Every output unit connects to every pixel and channel.
- A pattern at two positions gets two different sets of connections.
- Spatial neighborhoods are not built into the connectivity.

Image prior

Many useful patterns are local and recur at different positions. A convolution encodes these assumptions.

Model choice

This prior is helpful when it matches the task; images do not make dense layers mathematically invalid.

Even a small RGB image changes the dense-layer cost

For a 32×32 RGB image, the flattened input dimension is 3072.

$$P_{\text{dense}}(3072 \to 128) = (3072 + 1)128 = 393{,}344.$$

$$P_{\text{conv}}(3 \to 16, 3 \times 3) = 16(3 \cdot 9 + 1) = 448.$$

- These operations produce different output structures: a vector versus a spatial map.
- The counts reveal how full connectivity and shared local connectivity allocate parameters.
- A convolutional classifier still needs a head; do not compare one layer with a whole network.

Meaning

Specify the input, output, and connectivity before claiming that one architecture is smaller or more suitable.

An image tensor has spatial axes and channels

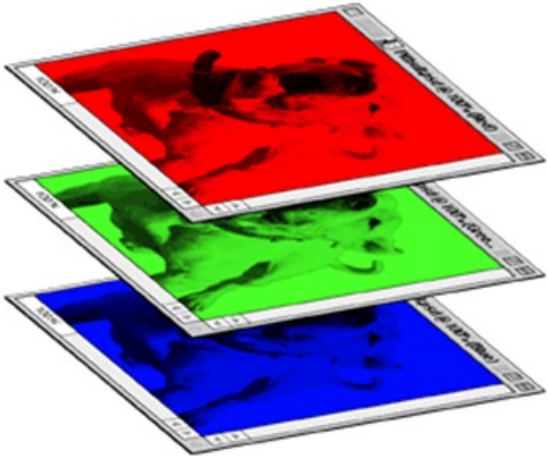
Axis	Meaning	Example
H	height, row coordinate	32 rows
W	width, column coordinate	32 columns
C	channels at each spatial position	3 for RGB
B	examples in a batch	8 images

- A common model layout is $B \times C \times H \times W$.
- Image libraries may store $H \times W \times C$; permute axes explicitly.
- RGB and BGR order are different channel meanings.
- A feature-map channel is a learned response, not necessarily a color.

Shape

8 RGB images of 32×32 give a model tensor of shape $(8, 3, 32, 32)$.

An image tensor has spatial axes and channels



HWC and CHW contain the same pixels in different axis orders

For two pixel positions A, B:

$$A = (R, G, B) = (10, 20, 30), \quad B = (40, 50, 60).$$

An $H \times W \times C$ layout groups channels at each pixel. A $C \times H \times W$ layout has channel slices:

$$R = (10, 40), \quad G = (20, 50), \quad B = (30, 60).$$

- Permuting axes preserves which color belongs to which pixel.
- Reshaping without the intended axis permutation can change that correspondence.
- Add the batch dimension after defining the per-image layout.

Meaning

An axis is a semantic coordinate. Correct element count alone does not prove an image tensor has the correct layout.

Preprocessing gives pixel values a defined meaning

For an 8-bit channel value $q \in \{0, \ldots, 255\}$:

$$x = q/255, \qquad x'_c = (x_c - \mu_c)/\sigma_c.$$

- Scaling changes the numerical range; normalization uses a specified channel mean and standard deviation.
- Use training statistics or the preprocessing expected by a pretrained model.
- Keep channel order, scaling, and normalization consistent at inference.
- Geometric augmentation must preserve the intended target or transform its coordinates too.

Pixel example

$q = 128$ gives $x \approx 0.502$. With $\mu = 0.5$, $\sigma = 0.25$, the normalized value is about 0.00784.

Resize and crop make different assumptions about geometry

An image has shape 100×200. Resizing it to 100×100 halves horizontal distances but leaves vertical distances unchanged.

- A circle becomes horizontally compressed unless aspect ratio is preserved.
- A centered crop keeps local geometry but may remove objects near the edges.
- Padding can fit a target canvas without the same deformation, but introduces a boundary region.

Detection labels

If width is scaled by 0.5, box x coordinates must also be scaled by 0.5. A crop additionally shifts coordinates and may truncate a box.

Meaning

Preprocessing changes both appearance and, for spatial tasks, target coordinates. Choose it from the intended task.

Spatial arrangement distinguishes an image from a bag of pixels

$$X = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \qquad X' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Both matrices contain three ones and six zeros, but one has a vertical line and the other a diagonal.

- A pixel histogram cannot distinguish these arrangements.
- Flattening retains values and their indices, but a dense layer does not impose local connections.
- A local filter can respond differently to nearby spatial patterns.

Connection

Representation choices must preserve information that matters for the target. Spatial order often matters for vision.

Spatial arrangement distinguishes an image from a bag of pixels



Two filters test different directional contrasts

Let

$$X = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}, \quad K_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, \quad K_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}.$$

$$\langle K_x, X \rangle = 3, \qquad \langle K_y, X \rangle = 0.$$

- The input changes across columns, so a left-right contrast filter responds.
- The rows are identical, so an upper-lower contrast filter cancels.

Meaning

Different channels can encode different local questions. Filter orientation and sign must be interpreted from its actual weights.

Local patterns recur at different positions



Read the example

- A small region can contain useful evidence such as a bird beak.
- The same pattern can appear in several image positions.
- One set of filter weights can test every position.

Two assumptions

Local connectivity limits what each unit sees; weight sharing reuses the same test across positions.

Locality and sharing can be counted separately on a tiny grid

A 4×4 single-channel input and a valid 3×3 filter produce four output positions.

Connections	Parameter count	Reason
Dense to four outputs	$4(16+1)=68$	all input pixels per output
Local without sharing	$4(9+1)=40$	independent local tests
Local with sharing	$9+1=10$	one reused filter

Meaning

Restricting the visible patch and tying parameters are two distinct choices. Shared parameters reduce freedom while still requiring four local computations.

Equivariance moves responses; invariance keeps a decision

For a stride-one shared convolution away from boundary effects:

$$\mathrm{Conv}(T_\Delta X) = T_\Delta \,\mathrm{Conv}(X).$$

Property	Meaning	Typical need
Translation equivariance	shift input, shift response map	localize the same pattern
Translation invariance	shift input, retain an output decision	classify an object independent of position

- Shared filters naturally support the first property.
- Pooling or global aggregation can encourage some tolerance to shifts.
- Padding, stride, cropping, and task requirements limit exact invariance.

A one-dimensional shift moves a shared-filter response

On an infinite zero-padded sequence, use $y_i = x_i - x_{i+1}$. Let x have one nonzero value $x_1 = 1$.

$$y_0 = -1, \quad y_1 = 1, \quad \text{other } y_i = 0.$$

Move the impulse to $x'_2 = 1$:

$$y'_1 = -1, \quad y'_2 = 1, \quad \text{other } y'_i = 0.$$

- The same response pattern shifts by one position.
- The response is not unchanged at each coordinate; it is equivariant.
- A finite image adds boundary conditions, so the exact statement needs that context.

Meaning

Shared local rules preserve the relationship between pattern position and response position, which is useful for localization.

A filter makes one local weighted sum at each position

For one channel, stride 1, no padding, and a 3×3 filter:

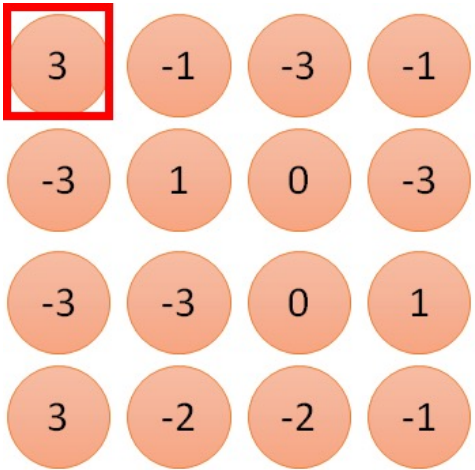
$$Y_{i,j} = b + \sum_{u=0}^{2} \sum_{v=0}^{2} K_{u,v} X_{i+u,j+v}.$$

- (i, j) is an output location; (u, v) is a location inside the filter.
- Each output reads one 3×3 input patch.
- The same nine weights and one bias are used at every output location.
- An activation such as ReLU is applied after this weighted sum.

Terminology

Most deep-learning libraries perform cross-correlation: the filter is not flipped. Mathematical convolution flips the kernel. Learned filters still serve the same modeling role.

A filter makes one local weighted sum at each position



Flipping the kernel changes this signed calculation

For the running patch

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 5 & 6 & 7 \\ 9 & 10 & 11 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}:$$

cross-correlation gives -6. Flipping K along both spatial axes gives

$$K_{\mathrm{flip}} = -K, \qquad \langle K_{\mathrm{flip}}, X \rangle = +6.$$

- Mathematical convolution uses the flipped indexing convention.
- Common neural-network layers use the unflipped convention.

Meaning

A hand calculation and a software result must use the same convention. Learning the kernel does not justify mixing conventions within one calculation.

A complete example: a 3x3 filter on a 4x4 input

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}, \quad b = 0.$$

- Stride 1, padding 0: two valid positions across and two down.
- For the top-left output, use rows 1–3 and columns 1–3 of X.
- The middle filter column contributes zero; compare left and right columns.

$$Y_{1,1} = (1 - 3) + (5 - 7) + (9 - 11) = -6.$$

Interpret the sign

The right side is brighter than the left in this patch; this filter assigns a negative response.

A contrast filter ignores a uniform offset when its weights sum to zero

For our kernel K, $\sum_{u,v} K_{u,v} = 0$.

$$\langle K, X + c\mathbf{1} \rangle = \langle K, X \rangle + c \sum K = \langle K, X \rangle.$$

- Adding 10 to every value in the worked patch leaves its response -6.
- A constant patch gives zero response with bias zero.
- This property belongs to this zero-sum kernel, not to every learned filter.

Another kernel

A kernel of nine ones responds to a uniform offset with an added $9c$.

Meaning

Specific weight constraints determine what changes a filter can ignore. Do not claim all CNN features are invariant to brightness.

Complete the feature map before applying activation

$$
\begin{aligned}
Y_{1,1} &= (1-3)+(5-7)+(9-11) = -6,\\
Y_{1,2} &= (2-4)+(6-8)+(10-12) = -6,\\
Y_{2,1} &= (5-7)+(9-11)+(13-15) = -6,\\
Y_{2,2} &= (6-8)+(10-12)+(14-16) = -6.
\end{aligned}
$$

$$
Y = \begin{bmatrix} -6 & -6 \\ -6 & -6 \end{bmatrix}, \qquad \mathrm{ReLU}(Y) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}.
$$

- A feature map keeps one response for every valid filter placement.
- The raw response and the activated response are different objects.
- Changing K to $-K$ makes the four responses $+6$ for this input.

The shared bias shifts every raw response before ReLU

Our raw 2×2 map without bias is uniformly -6.

Bias	Raw response	After ReLU	Effect
0	-6	0	inactive
5	-1	0	still inactive
7	1	1	active

- One bias is added at every location in this output channel.
- The same spatial contrast can cross the activation threshold after a bias change.
- Other output channels can have their own biases and filters.

Meaning

A feature-map value depends on the kernel, input patch, bias, and activation. Interpret all four before attributing a zero map to missing image structure.

A feature-map value is a response, not a label

- A vertical contrast filter measures a particular local pattern; another filter may respond to a different orientation or color combination.
- A high response means a strong match to that filter, not automatically a detected object.
- Several learned channels provide different evidence for later layers.
- Later layers combine local responses and larger context to support the task output.

From the worked example

All raw responses are -6 because every patch has the same horizontal contrast. The sign follows the chosen kernel, not a universal meaning of an edge.

Connection to Lecture 15

Hidden activations are intermediate features. A CNN organizes them by both channel and spatial coordinate.

Training learns useful filters from the task loss

- A filter is a set of trainable weights, not necessarily a hand-designed edge detector.
- Its update depends on which local patches contributed to prediction errors.
- Different channels can specialize or cooperate without each having a unique named concept.
- The target supervises the final output; intermediate interpretations require additional evidence.

Specific gradient

If a patch value is 4 and its output error is 0.2, that use contributes 0.8 to the corresponding kernel-weight gradient before combining other positions.

Meaning

The gradient links local measurements to task errors. A response becomes useful because of training and the surrounding network, not because it resembles a familiar drawing.

Shared weights sum gradient contributions over positions

For $Y_{i,j} = \sum_{u,v} K_{u,v} X_{i+u,j+v} + b$ and $G_{i,j} = \partial L / \partial Y_{i,j}$:

$$\frac{\partial L}{\partial K_{u,v}} = \sum_{i,j} G_{i,j} X_{i+u,j+v}, \qquad \frac{\partial L}{\partial b} = \sum_{i,j} G_{i,j}.$$

Our 4×4 example

If all four raw-output gradients are 1, the gradient for the top-left kernel weight is $1 + 2 + 5 + 6 = 14$, and the bias gradient is 4.

- One parameter participates in several output computations.
- Backpropagation adds every contribution, just as for shared branches in Lecture 15.
- If ReLU follows the convolution, multiply by its derivative before forming G.

Shared-filter gradients also flow to overlapping inputs

For $y_0 = k_0 x_0 + k_1 x_1$ and $y_1 = k_0 x_1 + k_1 x_2$, take $k = (1, 2)$ and output gradients $g = (3, 4)$:

$$\nabla_x L = (3,\ 2(3) + 1(4),\ 2(4)) = (3, 10, 8).$$

- The middle input is reused in both windows, so its contributions add.
- Kernel gradients separately sum the input-weight products over positions.
- For $x = (1, 2, 3)$, $\nabla_k L = (3(1) + 4(2),\ 3(2) + 4(3)) = (11, 18)$.

Meaning

Sharing and overlapping reuse create multiple paths. Backpropagation follows all paths rather than choosing only one window.

Output size counts valid filter placements

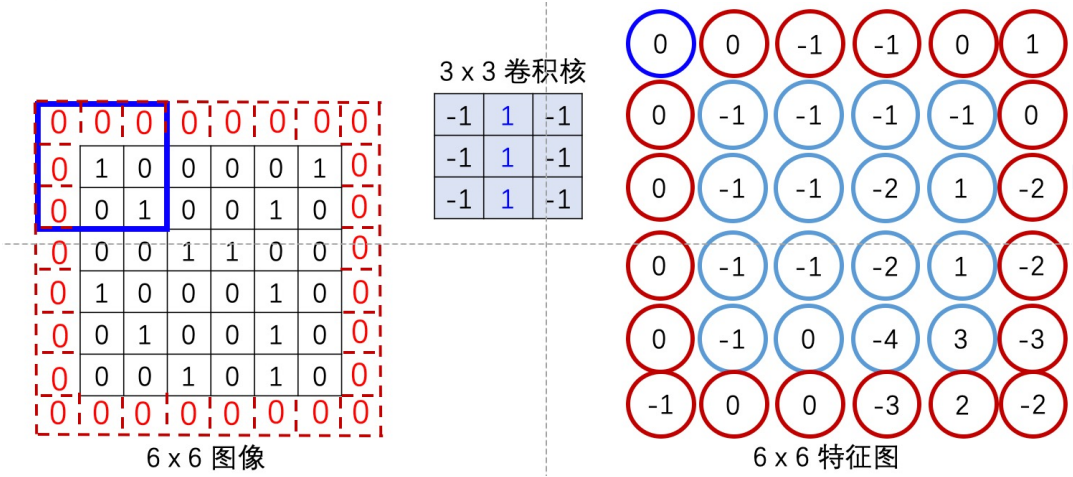
For each spatial axis, with input size n, kernel size k, stride s, and padding p on each side:

$$m = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1.$$

n	k	p	s	m
4	3	0	1	2
32	3	1	1	32
32	3	1	2	16
32	5	0	1	28

- Apply the formula separately to height and width.
- Here p means padding on each side; some references use total padding instead.
- Input channels do not appear in this spatial-size formula.

Output size counts valid filter placements



Calculate height and width separately for a rectangular input

For $H = 7$, $W = 9$, kernel 3×3, padding 1 per side, and stride 2:

$$H' = \lfloor (7 + 2 - 3)/2 \rfloor + 1 = 4,$$

$$W' = \lfloor (9 + 2 - 3)/2 \rfloor + 1 = 5.$$

With $C_{\text{in}} = 3$ and $C_{\text{out}} = 8$, one image gives shape $8 \times 4 \times 5$ and uses 224 parameters with biases.

- Spatial placement and channel count answer different shape questions.
- The same 224 parameters also process a larger image; output positions and compute would change.

Meaning

Do not assume square inputs or confuse the spatial-size formula with the parameter formula.

Padding preserves size but introduces a boundary rule

For odd k, stride 1, and dilation 1, using $p = (k-1)/2$ on each side preserves spatial size.

- A 3×3 kernel with $p = 1$ maps 32×32 to 32×32.
- Zero padding assumes missing surrounding values are zero.
- Other choices, such as reflection padding, encode different boundary behavior.
- Without padding, repeated layers shrink the spatial grid.

Two layers

$32 \to 30 \to 28$ for two valid 3×3 layers; $32 \to 32 \to 32$ with $p = 1$ at each layer.

Meaning of "same"

With stride greater than one, implementations may use asymmetric padding to achieve their documented output convention.

Zero padding changes an edge response even for a uniform image

Let a 3×3 image contain only ones, and use a 3×3 kernel of ones, padding 1, stride 1, bias 0:

$$Y = \begin{bmatrix} 4 & 6 & 4 \\ 6 & 9 & 6 \\ 4 & 6 & 4 \end{bmatrix}.$$

- The center reads nine observed ones.
- An edge reads six observed ones and three padded zeros.
- A corner reads four observed ones and five padded zeros.

Meaning

The input is uniform, yet zero-padded boundary responses differ. Padding defines what is assumed outside the observed image; it is not new evidence about the scene.

Period 1 checkpoint: one map, one filter, many uses

1. How many learned parameters does a single-channel 3×3 filter with one bias have?
2. For a 10×12 image, $k = 3$, $p = 0$, $s = 1$, what is the output size?
3. What is the difference between translation equivariance and invariance?
4. Why does a shared weight receive contributions from several output positions?

Answers

$9 + 1 = 10$ parameters; output 8×10. Equivariance moves the map, while invariance retains a decision. All uses of the same weight influence the loss.

Next period

The next filter reads several channels. Spatial size, channel count, parameter count, and compute cost must be tracked separately.

Exercise: output positions and shared parameters are independent counts

Input 5×5, one channel; two 3×3 output filters, no padding, stride 1, with biases.

1. Find the output tensor shape.
2. Count the learned parameters and total spatial filter applications.
3. Explain which quantity changes if the input becomes 7×7.

Solution

Output $2 \times 3 \times 3$; parameters $2(9 + 1) = 20$; filter applications $2(3 \cdot 3) = 18$. For 7×7, output is $2 \times 5 \times 5$ with 50 applications, while parameters remain 20.

Meaning

The number of uses changes with spatial resolution. The number of trainable values changes with the filter bank, not with every new placement.

Stride changes the sampling grid

$$Y_{i,j} = b + \sum_{u,v} K_{u,v} X_{si+u,sj+v} \quad \text{(no padding)}.$$

- Stride 1 uses neighboring filter starts; stride 2 skips every other start.
- Stride reduces the output grid and computation in later layers.
- The filter size and number of weights can remain unchanged.
- Fine position and small-object information can be lost.

Output calculation

For $n = 7$, $k = 3$, $p = 0$: stride 1 gives 5 positions, and stride 2 gives 3 positions. Starts are $0, 2, 4$.

Distinguish the quantities

Changing stride changes resolution; adding output filters changes channel count.

Stride can miss an alternating pattern after a one-pixel shift

Consider a one-dimensional sequence $x = (1, 0, 1, 0, 1, 0)$ and a 1-element filter of weight 1.

$$s = 2: \quad y = (x_0, x_2, x_4) = (1, 1, 1).$$

After shifting the pattern by one, $x' = (0, 1, 0, 1, 0, 1)$ gives $y' = (0, 0, 0)$.

- The sampling grid changed its alignment with the fine pattern.
- Larger stride is not automatically exact small-shift invariance.
- Smoothing before downsampling can reduce some aliasing, at a cost to fine detail.

Meaning

Reducing resolution is a substantive information choice. Assess it against the spatial scales that carry the target.

Dilation widens the sampled region without more kernel weights

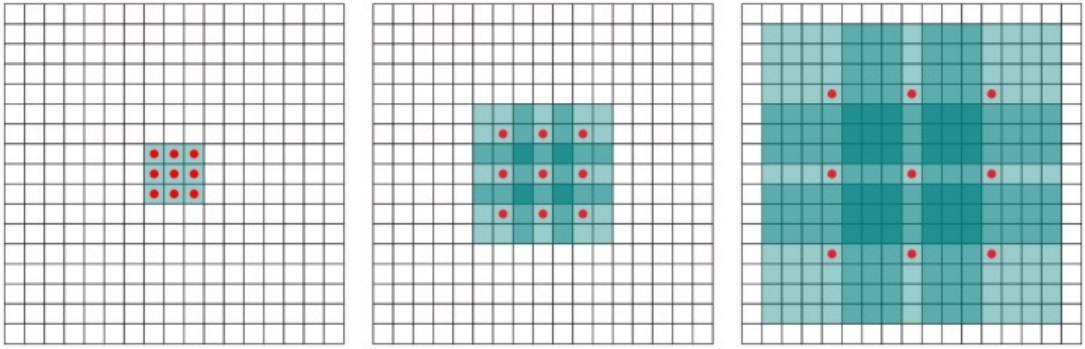
For dilation d, the effective spatial kernel size is

$$k_{\text{eff}} = d(k-1)+1, \qquad m = \left\lfloor \frac{n+2p-k_{\text{eff}}}{s} \right\rfloor + 1.$$

Kernel	Dilation	Effective extent	Weights per channel pair
3×3	1	3×3	9
3×3	2	5×5	9
3×3	3	7×7	9

- The filter samples spaced input locations, not every point in the larger extent.
- Useful when larger context is needed while preserving resolution.
- Sparse sampling can miss fine local patterns; a larger extent is not automatically better.

Dilation widens the sampled region without more kernel weights



(a) (b) (c)

Dilation and stride change different sets of coordinates

For a three-element filter with dilation 2 and stride 1:

$$y_i = k_0 x_i + k_1 x_{i+2} + k_2 x_{i+4}.$$

With stride 2, the first two windows start at 0 and 2:

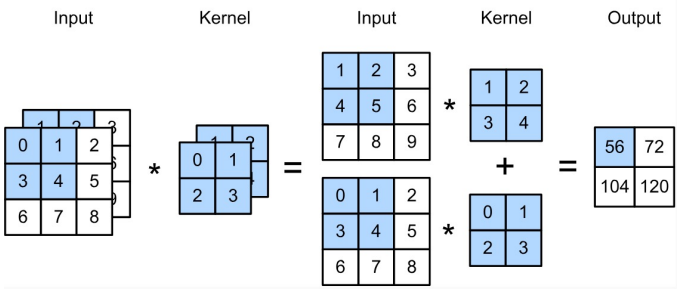
$$y_0 : k_0 x_0 + k_1 x_2 + k_2 x_4, \quad y_1 : k_0 x_2 + k_1 x_4 + k_2 x_6.$$

- Dilation sets spacing inside a filter placement.
- Stride sets spacing between placements.
- The same three learned weights can use either schedule.

Size

$n = 9, k = 3, d = 2, p = 0, s = 2$ gives effective size 5 and output length 3.

One output channel combines all input channels



Read the top-left output

- Channel 1 contributes 37.
- Channel 2 contributes 19.
- Their sum is 56 with bias 0.

The same channel-wise summation is repeated at each position.

$$Y_{i,j} = b + \sum_{c=1}^{C_{\text{in}}} \sum_{u,v} K_{c,u,v} X_{c,i+u,j+v}.$$

Meaning

A filter has one spatial slice per input channel; adding those slice responses gives one output channel.

A color combination can be learned at one pixel

For channels $(R, G, B) = (0.8, 0.2, 0.1)$, let a 1×1 filter use $(1, -0.5, -0.5)$ and bias 0:

$$a = 0.8 - 0.5(0.2) - 0.5(0.1) = 0.65.$$

For a gray pixel $(0.4, 0.4, 0.4)$, the response is 0.

- This filter compares red with the other channels at the same position.
- It does not read neighboring pixels or identify an object by itself.
- Standard spatial filters can combine this channel mixing with a larger local pattern.

Meaning

Channels are values at one location. A filter's input-channel slices jointly contribute to one output, even when spatial extent is only one pixel.

A filter bank creates several output channels

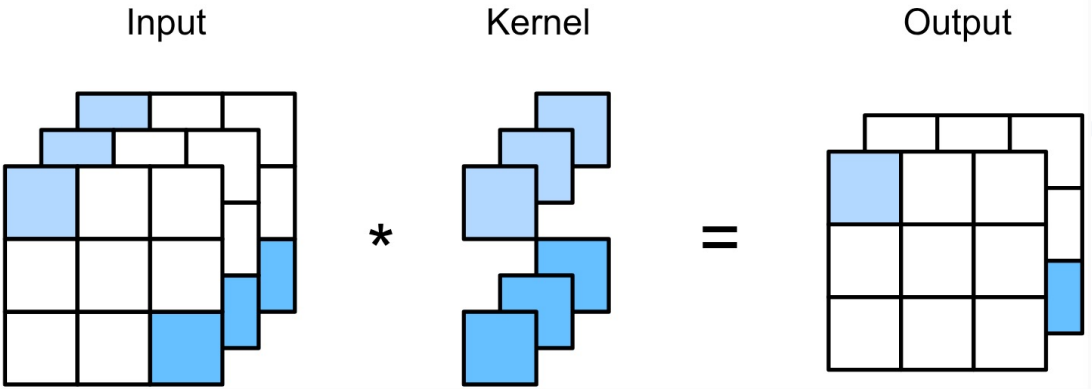
$$X \in \mathbb{R}^{B \times C_{\text{in}} \times H \times W}, \quad K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_H \times k_W}, \quad b \in \mathbb{R}^{C_{\text{out}}}.$$
$$Y \in \mathbb{R}^{B \times C_{\text{out}} \times H' \times W'}.$$

- One complete filter covers every input channel and produces one output map.
- C_{out} filters produce C_{out} different response maps.
- Spatial output size follows kernel, padding, stride, and dilation.

RGB to 16 channels

K has shape $(16, 3, 3, 3)$; with stride 1, padding 1, an input $(8, 3, 32, 32)$ gives output $(8, 16, 32, 32)$.

A filter bank creates several output channels



Two output filters use two complete channel mixtures

At one spatial position let $x = (2, 5)^\top$ and

$$K = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad b = (0, 1)^\top.$$

A 1×1 filter bank gives $y = Kx + b = (7, -2)^\top$.

- The first output adds both channels.
- The second subtracts them and adds its own bias.
- Each output filter covers both input channels; it is not assigned only one input color.

Count

$C_{\text{in}} = 2, C_{\text{out}} = 2, k = 1$ gives four weights and two biases. Repeating across an image does not create a new matrix at each pixel.

Convolution parameters depend on channels and kernel size

For standard convolution with one bias per output channel:

$$\#\text{parameters} = C_{\text{out}}(C_{\text{in}}k_H k_W + 1).$$

Layer	Weights	Biases	Total
$3 \to 16$, 3×3	$16 \cdot 3 \cdot 9 = 432$	16	448
$16 \to 32$, 3×3	$32 \cdot 16 \cdot 9 = 4{,}608$	32	4,640

- Input image height and width do not change this parameter count.
- Larger images still require more filter applications and more activation memory.
- Without biases, subtract C_{out} from the total.

Compute distinction

A layer with $32 \times 32 \times 16$ outputs performs about $32 \cdot 32 \cdot 16 \cdot 27$ multiply-accumulates for a 3-channel 3×3 kernel.

A layer can have few weights but many multiply-accumulates

For input $3 \times 32 \times 32$ and 16 output 3×3 filters with stride 1, padding 1:

$$P = 16(27 + 1) = 448, \qquad N_{\text{out}} = 16 \cdot 32 \cdot 32 = 16{,}384.$$

$$\text{multiply-accumulates} \approx 16{,}384 \cdot 27 = 442{,}368.$$

- The estimate excludes bias addition and activation work.
- Changing input to 64×64 keeps 448 parameters but roughly quadruples these operations.
- Activation memory also grows with the output grid.

Meaning

Parameter storage, arithmetic cost, and activation memory are different quantities. State which quantity a model-efficiency claim uses.

Local connections and sharing remove different parameters

Compare maps from $32 \times 32 \times 3$ to $32 \times 32 \times 16$.

Connectivity	Parameter reasoning	Total
Dense to every output	$(3072 + 1) \cdot 16384$	50,348,032
Local, no sharing	$16384(27 + 1)$	458,752
Shared 3×3 convolution	$16(27 + 1)$	448

- Locality restricts which input pixels affect an output.
- Sharing ties together weights at different spatial positions.
- Both restrictions encode image assumptions and reduce model freedom.

Fair reading

The dense case here produces a full output grid. It is not a comparison with a small classification head.

The same filter bank processes different image sizes

A standard layer has $C_{\text{in}} = 3, C_{\text{out}} = 8, k = 3, p = 1, s = 1$.

Input image	Output feature map	Parameter count
$3 \times 16 \times 16$	$8 \times 16 \times 16$	224
$3 \times 32 \times 32$	$8 \times 32 \times 32$	224
$3 \times 64 \times 64$	$8 \times 64 \times 64$	224

- Shared local computation does not require one fixed height and width.
- A later flattened dense head may require a specific flattened dimension.

Meaning

A flexible convolutional trunk does not automatically make every whole-network interface size-independent. Track the head too.

A 1x1 convolution mixes channels at each position

$$Y_{o,i,j} = b_o + \sum_{c=1}^{C_{\text{in}}} K_{o,c} X_{c,i,j}.$$

- It combines channels at one location; it does not directly mix neighboring locations.
- The same channel transformation is reused at every spatial position.
- It can reduce channels before an expensive spatial convolution.

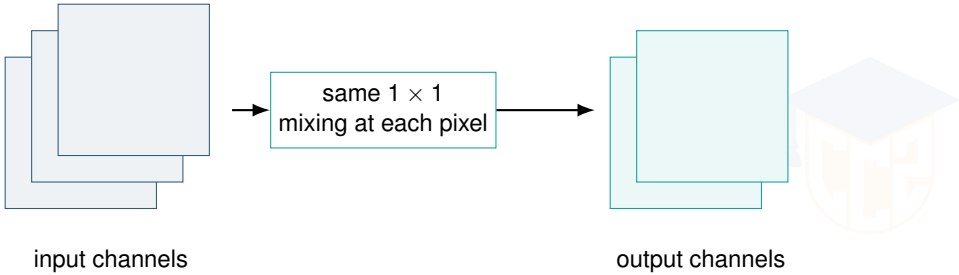
Two channels to one

At a pixel with channel values $(2, 5)$, weights $(0.5, -0.2)$ and bias 1 give $0.5(2) - 0.2(5) + 1 = 1$.

Parameter count

$64 \to 16$ channels use $16(64 + 1) = 1{,}040$ parameters with biases. Spatial size is unchanged for stride 1.

A 1x1 convolution mixes channels at each position



Channel reduction is a shared linear projection

For three input channels at one position, use

$$x = (2, 4, 6)^\top, \quad K = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0.5 & 0.5 \end{bmatrix}.$$

Then a bias-free 1×1 convolution gives $y = (-4, 5)^\top$.

- Three channel coordinates become two learned combinations.
- Neighboring positions are not mixed in this operation.
- A smaller channel representation can be sent to an expensive spatial layer.

Information limit

Reducing channels is not generally invertible. It aims to retain useful task information, not guarantee recovery of every original channel value.

Pooling summarizes a local feature-map patch

$$A = \begin{bmatrix} 1 & 3 & 2 & 0 \\ 4 & 2 & 1 & 5 \\ 0 & 1 & 6 & 2 \\ 3 & 2 & 4 & 1 \end{bmatrix}.$$

For 2×2 windows, stride 2:

$$\mathrm{MaxPool}(A) = \begin{bmatrix} 4 & 5 \\ 3 & 6 \end{bmatrix}, \qquad \mathrm{AvgPool}(A) = \begin{bmatrix} 2.5 & 2 \\ 1.5 & 3.25 \end{bmatrix}.$$

- Pooling operates separately on each channel and usually has no learned weights.
- Max pooling keeps the strongest response; average pooling keeps the local mean.
- The spatial grid shrinks, so exact within-window position is lost.

Task tradeoff

Downsampling can help classification efficiency but can hurt localization of small structures.

Pooling summarizes a local feature-map patch



迷川浩浩(CSDN)
https://blog.csdn.net/qq_21190081

Overlapping pooling windows reuse feature-map values

For

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 9 & 6 \\ 7 & 8 & 5 \end{bmatrix},$$

2×2 max pooling with stride 1 gives

$$Y = \begin{bmatrix} 9 & 9 \\ 9 & 9 \end{bmatrix}.$$

- The center belongs to all four windows and is the unique maximum in each.
- Stride 1 creates overlapping windows; stride 2 would use a different set of placements.
- Repeated output values do not mean four separate stored input maxima exist.

Meaning

The forward selection determines the backward routes. A reused input may receive several contributions from overlapping outputs.

Pooling gradients follow the forward selection

For patch $\begin{bmatrix} 1 & 3 \\ 4 & 2 \end{bmatrix}$ and incoming output gradient $g = 2$:

$$\nabla_A L \text{ through max pooling} = \begin{bmatrix} 0 & 0 \\ 2 & 0 \end{bmatrix},$$

$$\nabla_A L \text{ through average pooling} = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}.$$

- A unique maximum receives the max-pooling gradient; other values receive zero through this path.
- Average pooling distributes the gradient equally across the window.
- Overlapping windows add contributions at shared input positions.
- Ties in max pooling use an implementation-defined selection or subgradient convention.

Pooling gradients follow the forward selection



迷川浩浩(CSDN) https://blog.csdn.net/qq_21190081

Overlapping max-pooling gradients accumulate at the shared maximum

For the 3×3 map from the overlapping-pooling example, let

$$G = \frac{\partial L}{\partial Y} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

The center was selected by all four windows, so

$$\frac{\partial L}{\partial A} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

- $1 + 2 + 3 + 4 = 10$ is accumulated at the one shared input location.
- The unique-max assumption avoids ambiguity from ties.
- Average pooling would distribute each window contribution instead, then also add overlaps.

Meaning

Backward routing must match the actual forward selections. Overlap changes contribution counts even though pooling has no learned parameters.

Receptive field is an input region, not a channel count

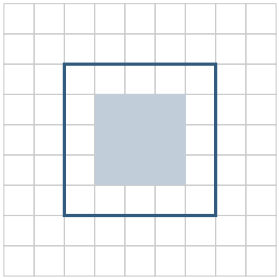
- The receptive field of one activation is the region of the original input that can influence it.
- A first-layer 3×3 convolution reads a 3×3 region.
- A second 3×3, stride-one layer combines neighboring first-layer responses and reaches a 5×5 input region.
- More channels provide more response types at the same positions; they do not by themselves enlarge the spatial receptive field.

$$3 \times 3 \longrightarrow 5 \times 5 \longrightarrow 7 \times 7 \quad \text{for three stride-one } 3 \times 3 \text{ layers.}$$

Theoretical versus effective

Being connected to an input region does not mean every pixel in that region has equally strong influence after training.

Receptive field is an input region, not a channel count



two 3×3, stride 1

a 5×5 receptive field

Trace two local layers back to the original input

For two stride-one valid 3×3 layers, a second-layer output at $(0, 0)$ reads first-layer positions $(0 \ldots 2, 0 \ldots 2)$.

- First-layer position $(0, 0)$ reads original rows and columns $0 \ldots 2$.
- First-layer position $(2, 2)$ reads original rows and columns $2 \ldots 4$.
- The union of all nine first-layer receptive fields covers original rows and columns $0 \ldots 4$.

$$r_2 = 3 + (3 - 1) \cdot 1 = 5.$$

Meaning

The 5×5 extent arises from overlapping regions. It is neither $3 + 3$ nor $3 \cdot 3$ along each axis, and more channels alone do not change these coordinates.

Track receptive-field size and spacing through a stack

Start with receptive field $r_0 = 1$ and spacing $j_0 = 1$. For layer l:

$$r_l = r_{l-1} + (k_l^{\text{eff}} - 1) j_{l-1}, \qquad j_l = j_{l-1} s_l.$$

Layer	k	s	r	j
Input	–	–	1	1
Conv	3	1	3	1
Pool	2	2	4	2
Conv	3	1	8	2
Pool	2	2	10	4

Read the final row

Each activation can cover a 10×10 original-image region; neighboring activations are four input pixels apart.

Boundary detail

Padding affects coordinates and edge behavior; this table tracks interior extent and spacing.

Combine stride and dilation in a receptive-field calculation

First layer: $k = 3, s = 2, d = 1$, so $r_1 = 3, j_1 = 2$. Second layer: $k = 3, s = 1, d = 2$, so $k_{\text{eff}} = 5$:

$$r_2 = 3 + (5 - 1)2 = 11, \qquad j_2 = 2.$$

- The second layer spans five positions of the first grid, with gaps inside its kernel.
- Adjacent first-grid positions are two original pixels apart; dilation samples positions four original pixels apart.
- The extent is eleven original pixels, while neighboring second-layer outputs are still two pixels apart.

Meaning

Receptive-field extent and output spacing are separate tracked quantities. Use effective kernel size for dilation and prior spacing for the increment.

Flattening and global averaging create different heads

For a final feature map with shape $C \times H \times W$:

Operation	Output	What changes
Flatten	CHW values	retains location-specific coordinates
Global average pooling	C values	averages each channel over the grid

Head size

$32 \times 8 \times 8$ to ten classes: flatten then dense uses $2048(10) + 10 = 20{,}490$ parameters; global average then dense uses $32(10) + 10 = 330$.

- Global averaging reduces parameters and removes explicit spatial coordinates from the head.
- It is useful for many classification tasks, but detailed localization needs spatial outputs.

Global average pooling sends a uniform within-channel gradient

For one 2×3 channel:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \qquad q = \frac{1}{6} \sum A = 3.5.$$

If the incoming gradient is $\partial L / \partial q = 12$:

$$\frac{\partial L}{\partial A_{i,j}} = 12/6 = 2 \quad \text{at every position.}$$

- All positions contribute equally to this averaging operation.
- Different channels remain separate and may receive different gradients from the head.
- A later linear head can still weight channels differently.

Meaning

Global averaging aggregates spatial locations, not all channels into one number. The gradient follows exactly the weights of that aggregation.

A small classifier: calculate every stage

Stage	Operation	Output per image	Parameters
Input	RGB image	$3 \times 32 \times 32$	0
1	Conv $3 \to 16$, $k = 3, p = 1$; ReLU	$16 \times 32 \times 32$	448
2	MaxPool $k = 2, s = 2$	$16 \times 16 \times 16$	0
3	Conv $16 \to 32$, $k = 3, p = 1$; ReLU	$32 \times 16 \times 16$	4,640
4	MaxPool $k = 2, s = 2$	$32 \times 8 \times 8$	0
5	Global average pooling	32	0
6	Dense $32 \to 10$	10 logits	330

Total

There are $448 + 4{,}640 + 330 = 5{,}418$ learned parameters.

Connection to Lecture 15

Use multiclass cross-entropy on the ten logits; backpropagation trains both filters and the output head.

A flattened head changes when the input resolution changes

The two-pool trunk maps 32×32 inputs to $32 \times 8 \times 8$, but 64×64 inputs to $32 \times 16 \times 16$.

$$D_{\mathrm{flat}} = 2048 \quad \text{versus} \quad 8192.$$

For ten classes, dense-head counts are 20,490 and 81,930 with biases.

- A fixed $2048 \to 10$ layer cannot directly accept the larger flattened vector.
- Global averaging produces 32 values in both cases, allowing the same $32 \to 10$ head.
- A compatible interface does not guarantee accuracy under an image-scale change.

Meaning

Track both computation shape and task behavior when changing input resolution.

Head design can dominate the parameter count

For the same convolutional trunk, compare:

Head	Trunk parameters	Head parameters	Total
Flatten $2048 \rightarrow 10$	5,088	20,490	25,578
Global average $32 \rightarrow 10$	5,088	330	5,418

- The convolutional feature maps are identical up to the last pooling stage.
- The larger dense head can exploit position-specific features but adds more adjustable parameters.
- Global averaging provides a smaller head by aggregating spatial responses.
- Select using the task, data size, and validation results rather than parameter count alone.

The code follows the shape table

```
model = torch.nn.Sequential(
    torch.nn.Conv2d(3, 16, 3, padding=1),
    torch.nn.ReLU(),
    torch.nn.MaxPool2d(2),
    torch.nn.Conv2d(16, 32, 3, padding=1),
    torch.nn.ReLU(),
    torch.nn.MaxPool2d(2),
    torch.nn.AdaptiveAvgPool2d(1),
    torch.nn.Flatten(),
    torch.nn.Linear(32, 10))
logits = model(images)  # images: B x 3 x 32 x 32
loss = torch.nn.functional.cross_entropy(logits, labels)
```

Check the interface

Logits have shape $B \times 10$; labels are integer class indices with shape B. Count parameters and compare with 5,418.

Period 2 checkpoint: four quantities, four different questions

1. A $3 \to 8$ convolution with 3×3 kernels and biases has how many parameters?
2. For input 28×28, $k = 3, p = 1, s = 2$, what is the output size?
3. Does increasing output channels from 8 to 16 enlarge the spatial receptive field?
4. Why must the input size of a dense head match the final feature map?

Answers

$8(3 \cdot 9 + 1) = 224$ parameters; output 14×14; more channels alone do not enlarge the receptive field. A flattened $C \times H \times W$ map contains CHW values.

Next period

We will use these quantities to understand architecture choices, rather than memorizing model names.

Exercise: remove convolution biases and recalculate the total

Use our two-convolution classifier, but set both convolution biases to false while retaining the ten-class head bias.

1. Do output tensor shapes change?
2. Recalculate all learned parameter counts.
3. Does removing a bias also remove the filter's spatial weights?

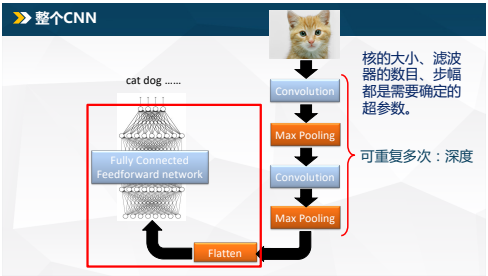
Solution

Shapes do not change. The convolution counts become 432 and 4608; the head remains 330. Total 5370, exactly 48 fewer than 5418.

Meaning

Configuration details affect exact counts. State which biases or normalization parameters are included when comparing a hand calculation with a model summary.

A hierarchy combines local evidence into larger context



Follow the dimensions

- Convolutions produce spatial response maps.
- Activations introduce nonlinear combinations.
- Downsampling reduces grid resolution.
- Later layers cover larger input context.
- The head converts features to the required prediction.

Interpretation boundary

A hierarchy enables feature composition; it does not guarantee that every channel has one named human concept.

Later layers mix both feature types and neighboring positions

Two one-dimensional input channels are $A = (1, 0, 1)$ and $B = (0, 1, 0)$. One output filter uses $K_A = (1, 1)$ and $K_B = (1, -1)$:

$$y_0 = (1 + 0) + (0 - 1) = 0, \qquad y_1 = (0 + 1) + (1 - 0) = 2.$$

- The same channel mixture is applied at each valid position.
- A response depends on which patterns occur together in the local window.
- Later features are combinations of earlier responses, not merely a larger copy of one early channel.

Meaning

A feature hierarchy composes spatial and channel information. Its usefulness still depends on what the training task rewards.

Classic CNN families teach different design lessons

Family	Core design lesson	Connection to our calculation
LeNet	alternate local feature extraction and down-sampling	small digit-image classifier
AlexNet	train a larger CNN with ReLU and suitable data/compute	scale and training choices matter
VGG	stack simple small kernels in repeated stages	receptive field grows through composition
Inception	parallel receptive-field scales with channel reduction	shape-compatible branches
ResNet	add an identity or projection path to learned features	residual correction and gradient paths

Read architecture diagrams

Ask what each stage changes in resolution, channels, context, parameters, and training behavior.

A LeNet-style example has a fully traceable shape sequence

For a simplified digit classifier (not an exact historical implementation):

Operation	Output per image	Parameters with biases
Input	$1 \times 28 \times 28$	0
Conv $1 \to 6, k = 5$	$6 \times 24 \times 24$	156
Pool $2, s = 2$	$6 \times 12 \times 12$	0
Conv $6 \to 16, k = 5$	$16 \times 8 \times 8$	2416
Pool $2, s = 2$	$16 \times 4 \times 4$	0
Dense $256 \to 120 \to 84 \to 10$	10 logits	$30840 + 10164 + 850$

Total

The simplified model uses 44,426 parameters. Its large dense part illustrates why the head can dominate the count.

Two small kernels can replace one larger spatial extent

For stride-one layers with C input and output channels and biases omitted:

Option	Receptive-field extent	Weights	Nonlinear stages
One 5×5 convolution	5×5	$25C^2$	one
Two 3×3 convolutions	5×5	$18C^2$	two if activated

For $C = 64$

One large kernel uses 102,400 weights; two small kernels use 73,728.

- Stacking grows context while allowing an extra nonlinearity.
- Two activated layers are not generally the same function as one large convolution.
- The comparison assumes the intermediate channel count is also C.

The extra activation changes what two small layers compute

Suppose a first spatial layer produces two local values $(-1, 2)$ and the next layer adds them.

$$\text{without middle activation: } (-1) + 2 = 1,$$

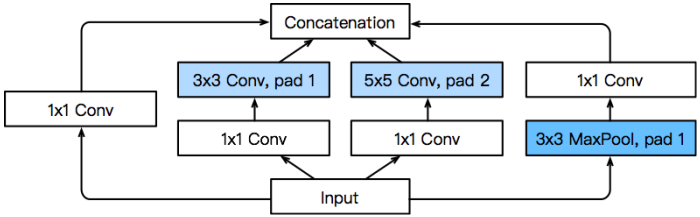
$$\text{with middle ReLU: } 0 + 2 = 2.$$

- Two stacked linear kernels can be represented by a larger effective linear operation under suitable conditions.
- A nonlinear activation between them changes the input-dependent computation.
- Equal receptive-field extent does not imply equal function or prediction.

Meaning

The two-3×3 versus one-5×5 comparison concerns both parameter cost and nonlinear composition. State which property is being compared.

Inception combines branches with matching spatial grids



Read from bottom to top

- Different branches read different spatial scales.
- Padding keeps branch heights and widths compatible.
- 1×1 layers can reduce channels before costly kernels.
- Concatenation adds channel counts, not spatial sizes.

Output channels

Branches with $16, 24, 8, 8$ channels on the same $H \times W$ grid concatenate to $56 \times H \times W$.

An Inception concatenation needs matching spatial dimensions

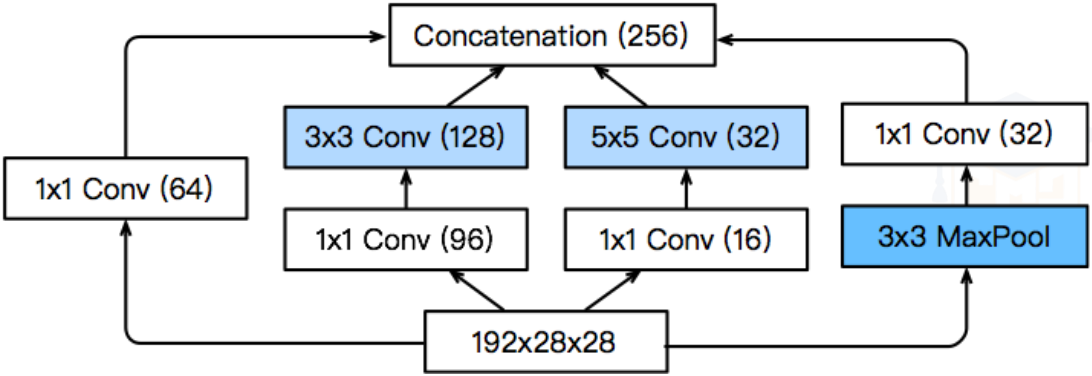
For input $64 \times 28 \times 28$, four stride-one branches can produce:

Final branch operation	Padding	Output shape
1×1 to16 channels	0	$16 \times 28 \times 28$
3×3 to24 channels	1	$24 \times 28 \times 28$
5×5 to8 channels	2	$8 \times 28 \times 28$
3×3 MaxPool, then 1×1 to 8	pool $s = 1, p = 1$	$8 \times 28 \times 28$

Output

Concatenating channels produces $56 \times 28 \times 28$. An unpadded 3×3 branch would have size 26×26 and could not directly join this grid.

An Inception concatenation needs matching spatial dimensions



A bottleneck reduces expensive channel mixing

Compare a $128 \to 128$ 3×3 layer with a bottleneck; biases omitted.

Operation	Channel change	Weights
Direct 3×3	$128 \to 128$	$9 \cdot 128^2 = 147{,}456$
Reduce 1×1	$128 \to 32$	4,096
Spatial 3×3	$32 \to 32$	9,216
Expand 1×1	$32 \to 128$	4,096
Bottleneck total		17,408

- Only the spatial middle layer uses the small channel width.
- The final grid can have the original channel count again.
- Channel compression restricts the intermediate representation; verify that the task retains useful information.

Bottlenecks reduce computation at every spatial position

For a 28×28 grid and biases omitted, the previous direct and bottleneck weights are 147,456 and 17,408.

$$\text{direct MACs} \approx 784 \cdot 147{,}456 \approx 115.61 \text{ million},$$

$$\text{bottleneck MACs} \approx 784 \cdot 17{,}408 \approx 13.65 \text{ million}.$$

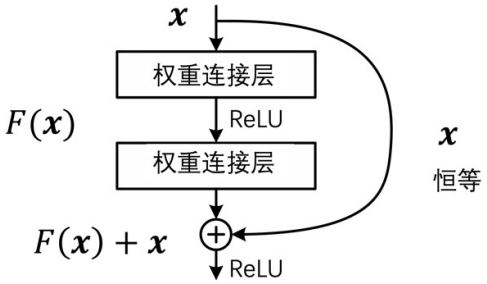
- The estimate assumes all three bottleneck layers retain this grid size.
- Nonlinearities, memory access, and hardware scheduling add costs not captured here.
- The compressed intermediate channels change the representational constraint.

Meaning

A weight saving repeats across all positions, but lower arithmetic count is not automatically the same ratio of measured runtime or guaranteed accuracy.

A residual block learns a correction to its input

$$u = x + F(x; \theta).$$



- The shortcut carries the input directly.
- The learned branch adds a correction.
- Setting F close to zero preserves the input before any following activation.
- Addition requires matching channel and spatial shapes.

Scalar intuition

If $x = 2$ and $F(x) = 0.3$, the pre-activation sum is 2.3.

Block variants

The original post-activation diagram applies ReLU after the sum; other residual designs place activations differently.

A zero residual preserves the sum before any following activation

Let $x = (-2, 3)$ and $F(x) = 0$:

$$u = x + F(x) = (-2, 3).$$

For a post-activation block:

$$\mathrm{ReLU}(u) = (0, 3),$$

while a block with no activation after the sum retains u itself.

- The shortcut contributes the input to the sum.
- A subsequent activation can still change negative components.
- Activation placement is part of the block definition.

Meaning

Do not silently call every entire residual block an identity when its residual branch is zero. Read the operations after addition too.

The identity path contributes an explicit gradient term

For the residual sum $u = x + F(x)$:

$$\frac{\partial u}{\partial x} = I + J_F(x), \qquad \frac{\partial L}{\partial x} = (I + J_F(x))^\top \frac{\partial L}{\partial u}.$$

Scalar example

If $F(x) = 0.2x$ and the incoming gradient is $g = 3$, then $dL/dx = (1 + 0.2)3 = 3.6$. The two paths contribute 3 and 0.6.

- The shortcut adds a direct path that does not multiply through every weight in F.
- This can make deep optimization easier, but it does not guarantee nonzero or well-scaled gradients.
- A following activation or a projection shortcut adds its own derivative factors.

An identity path does not prevent every derivative cancellation

For $u = x + \theta x = (1 + \theta)x$ with $\theta = -1$ and $x = 2$:

$$u = 0, \quad \frac{\partial u}{\partial x} = 1 + \theta = 0, \quad \frac{\partial u}{\partial \theta} = x = 2.$$

- Input sensitivity cancels between the two paths.
- The branch parameter can still receive a nonzero gradient from the same output.
- A direct route is useful for optimization, but it is not a guarantee about every derivative at every point.

Meaning

State which derivative is being discussed. Input, activation, and parameter gradients are related but not interchangeable quantities.

Use a projection when residual shapes differ

If x has shape $64 \times 32 \times 32$ but $F(x)$ has shape $128 \times 16 \times 16$, direct addition is invalid.

$$u = P(x) + F(x),$$

where P can be a 1×1 convolution with $64 \to 128$ channels and stride 2.

- The projection aligns channel count and spatial resolution.
- This shortcut is learned, not an exact identity.
- Both paths must produce the same final shape before addition.

Projection weights

With no bias, $128 \cdot 64 = 8{,}192$ weights. The stride changes the grid; the 1×1 weights change channels.

Contrast with concatenation

Adding 128-channel maps returns 128 channels; concatenating them returns 256 channels.

A projected shortcut has both a shape change and learned weights

Projection P: 1×1 convolution, $64 \to 128$ channels, stride 2, no bias.

$$64 \times 32 \times 32 \longrightarrow 128 \times 16 \times 16, \qquad P \text{ has } 8192 \text{ weights.}$$

- Each output position mixes 64 input channels.
- Stride selects a coarser spatial grid; it is not caused by the extra output channels.
- A main branch ending at the same shape can be added elementwise.
- With a projection bias, add another 128 parameters.

Meaning

A learned projection is different from a parameter-free identity shortcut. Both shape compatibility and the extra parameters belong in the architecture description.

Depthwise separable convolution restricts channel connections

For $C_{\text{in}} = 32$, $C_{\text{out}} = 64$, $k = 3$, with biases omitted:

Operation	Connection pattern	Weights
Standard convolution	spatial and channel mixing together	$9 \cdot 32 \cdot 64 = 18{,}432$
Depthwise stage	one spatial filter per input channel	$9 \cdot 32 = 288$
Pointwise stage	1×1 channel mixing	$32 \cdot 64 = 2{,}048$
Separable total	depthwise then pointwise	2,336

- The depthwise output initially has 32 channels.
- The pointwise stage mixes them to produce 64 output channels.
- Fewer weights can help efficient models, but the restricted form is not identical to every standard convolution.

Grouped and depthwise convolutions restrict different connections

For $32 \to 64$ channels and 3×3 kernels, biases omitted:

Operation	Weights	Channel interaction
Standard	18,432	all inputs per output
Grouped, $G = 4$	$64(32/4)9 = 4608$	within four groups
Depthwise then pointwise	$288 + 2048 = 2336$	spatial separation, then mixing

- Group counts must be compatible with input and output channel counts.
- A grouped layer alone does not mix information across its groups.

Meaning

Efficient convolution changes the allowed interactions. Parameter reduction should be interpreted alongside those representational restrictions.

The prediction target determines the vision head

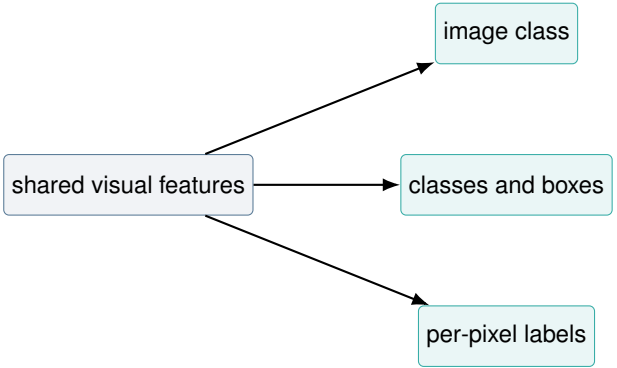
Task	Prediction	Information the head needs
Classification	one class per image	global evidence
Detection	classes and bounding boxes	object identity and location
Semantic segmentation	one class per pixel	dense spatial details and context

- A shared convolutional backbone can support several tasks.
- Global averaging is natural for many classification heads.
- Detection and segmentation must retain or recover spatial information.
- Labels and losses differ; changing only the final neuron count is often insufficient.

Concrete contrast

“There is a car” does not answer “Where is it?” or “Which pixels belong to it?”

The prediction target determines the vision head



A board-game policy needs location-specific outputs

A 19×19 Go board can be encoded with channels for stones and relevant state. A move-policy head produces scores on board positions:

$$\text{spatial features } C \times 19 \times 19 \longrightarrow 19 \times 19 \text{ move logits.}$$

- Nearby stone patterns motivate local shared computation.
- The answer is a location, so aggressive pooling can remove required positional detail.
- Legal-move masking and game state are part of the prediction definition.
- Image-like input does not imply that image-classification pooling is suitable.

Meaning

Read the task before reusing a familiar CNN. Classification of the whole board and selection of a move require different outputs.

Classification aggregates evidence for one image label

feature map $C \times H \times W \longrightarrow$ pooled vector $C \longrightarrow K$ logits.

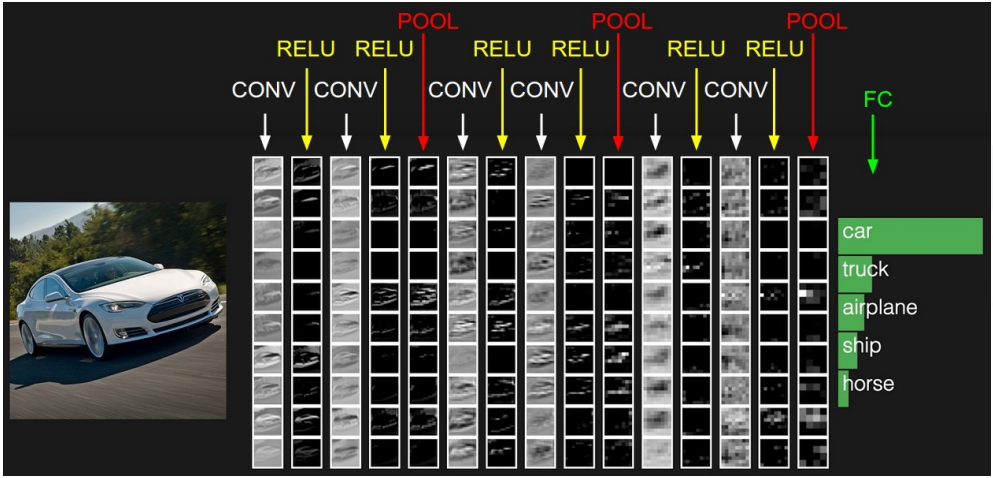
$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad L = -\log p_y.$$

- The backbone and head are trained together using the image label.
- Evaluate unseen images from the intended use population.
- Use a confusion matrix to identify which classes are mixed up.
- High image-level accuracy does not demonstrate correct localization.

Binary interpretation

A model with high confidence for “car” may be using background cues. Compare errors across backgrounds before claiming the feature is the object itself.

Classification aggregates evidence for one image label



High accuracy can conceal a completely missed minority class

In 100 images, 90 are background and 10 contain the target. Predicting background for every image gives:

True class	Predicted background	Predicted target
Background	90	0
Target	10	0

$$\text{accuracy} = 90/100 = 0.9, \qquad \text{target recall} = 0/10 = 0.$$

- The high total score comes entirely from the common class.
- Per-class errors reveal that the target task failed.

Meaning

A vision metric must reflect the errors the task cares about. Inspect class-specific behavior rather than reporting one attractive aggregate.

Detection predicts both a category and a box

For a bounding box (x_1, y_1, x_2, y_2), a detector needs object confidence, class information, and coordinate prediction.

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Box calculation

Boxes $(0, 0, 2, 2)$ and $(1, 1, 3, 3)$ have areas 4 and 4, intersection 1, union 7, so $\mathrm{IoU} = 1/7 \approx 0.143$.

- Correct class alone does not guarantee a well-located box.
- Multiple objects require multiple predictions and a matching strategy.
- Downsampling can make small objects hard to localize; several feature scales can help.

Task link

The output must retain coordinates, not only a globally pooled image label.

An oversized box can contain the object and still have low IoU

Let the true box be $(1, 1, 3, 3)$ with area 4, and the predicted box $(0, 0, 4, 4)$ with area 16.

$$|A \cap B| = 4, \qquad |A \cup B| = 16, \qquad \text{IoU} = 4/16 = 0.25.$$

- Containing the full object is not the same as locating its boundary well.
- A prediction identical to the true box gives IoU 1.
- A disjoint box gives IoU 0, even if its category name is correct.

Meaning

Localization quality includes both coverage and excess area. Detection needs category and position evidence, not only image-level confidence.

Segmentation predicts classes on a spatial grid

A semantic segmentation head produces logits with shape

$$B \times K \times H \times W,$$

and the target commonly has shape $B \times H \times W$ with one class index per pixel.

- A low-resolution feature map contains context but lacks the full output grid.
- Upsampling or a decoder restores spatial resolution.
- Skip features can combine coarse context with finer details.
- Pixel accuracy can be misleading when one background class dominates.

Class IoU

For one class, $TP = 60$, $FP = 20$, $FN = 20$ gives $\text{IoU} = 60/(60 + 20 + 20) = 0.6$. Mean IoU averages the chosen class IoUs.

Pixel accuracy and class IoU can rank the same mask differently

In a 4×4 mask, the target class has 4 pixels. A prediction has $TP = 3$, $FP = 3$, $FN = 1$, $TN = 9$:

$$\text{pixel accuracy} = (3 + 9)/16 = 0.75,$$

$$\text{IoU}_{\text{target}} = 3/(3 + 3 + 1) = 3/7 \approx 0.429.$$

- Predicting only background also gets $12/16 = 0.75$ accuracy, but target IoU is zero.
- The two predictions have equal accuracy while treating the target very differently.
- Each class's IoU and the chosen averaging convention should be explicit.

Meaning

For dense spatial labels, inspect the class-specific overlap that the application needs, not only the fraction of correctly labeled pixels.

Transfer learning reuses features, then adapts to the task

1. Start from a backbone trained on a relevant large dataset.
2. Replace the prediction head for the new target and label count.
3. Initially train the new head, or fine-tune selected layers with a suitable learning rate.
4. Keep the pretrained model's expected preprocessing unless intentionally changing it.
5. Compare with a simpler baseline on held-out data from the intended task.

Why it can help

Earlier training provides useful visual features when the new labeled dataset is small.

Why it can fail

Different image modalities, scales, or task cues can make inherited features less suitable. Pretraining does not remove the need for task-specific evaluation.

Retain the reasoning, not only the architecture names

1. Local filters test nearby patterns; sharing reuses the same test across positions.
2. Spatial size depends on kernel, padding, stride, and dilation.
3. Channels describe response types; parameter counts include input channels, output filters, and biases.
4. Downsampling changes position resolution, while receptive field tracks input context.
5. Residual paths and bottlenecks change computation and training routes in explainable ways.
6. The task decides whether to aggregate the grid or preserve spatial outputs.

Our small classifier

Two convolutions, two pools, global averaging, and a ten-class head use 5,418 parameters.

Next lecture

Training choices must help this concrete network learn from data; they cannot replace a suitable task formulation.

Exit exercise: design and calculate a five-class CNN

Input: $3 \times 64 \times 64$.

1. Conv $3 \to 16$, $k = 3, p = 1, s = 2$; ReLU.
2. Conv $16 \to 32$, $k = 3, p = 1, s = 1$; ReLU.
3. Global average pooling; dense $32 \to 5$ logits.

Calculate

Write all output shapes, count parameters including biases, and find the second convolution's receptive-field extent.

Result	Value	Reason
Feature maps	$16 \times 32 \times 32 \to 32 \times 32 \times 32$	stride then channel change
Parameters	$448 + 4{,}640 + 165 = 5{,}253$	two convolutions and one head
Receptive field	7×7	$r_1 = 3, j_1 = 2$; $r_2 = 3 + 2(2) = 7$

The same parameter count can support a different output structure

Replace the five-class global-average classifier head by a 1×1 spatial head $32 \to 5$ with biases.

$$32 \times 32 \times 32 \longrightarrow 5 \times 32 \times 32.$$

The head still uses $(32 + 1)5 = 165$ parameters, so the total remains 5253.

- It now produces five logits at every coarse-grid position, rather than five logits for the image.
- A segmentation target at 64×64 needs output-grid alignment, such as an appropriate upsampling/decoder design.
- Equal parameter counts do not imply equal tasks, spatial information, or loss definitions.

Meaning

Architecture should be described by what is computed and predicted, not only by a total number of parameters.